

Assignment K-Nearest Neighbor (KNN) AND K-Means Clustering

CSE 3812
Artificial Intelligence Assignment
Department of CSE
United International University

May 24, 2026

Objective 1

The objective of this offline is to implement the **K-Nearest Neighbors (KNN) classification algorithm from scratch** and analyze how the choice of K affects classification performance.

Dataset Preparation

Use a publicly available labeled dataset suitable for classification. link: [Click here](#)

- The dataset must contain:
 - Numeric feature columns
 - One discrete class label column
- Split the dataset into:
 - Training set (e.g., 70%)
 - Test set (e.g., 30%)

Algorithm with Code Hints

1. Choose the value of K (number of nearest neighbors)
Example: $K = 3$
2. Load the dataset into memory
Example: `data = np.loadtxt("dataset.txt")`

3. Separate features (X) and labels (y)
4. Split data into training and test sets
5. For each test sample:
 - a. Compute the distance between the test point and all training points
Hint: use Euclidean distance
 - b. Sort the distances in ascending order
 - c. Select the K nearest neighbors
 - d. Perform majority voting on their class labels
 - e. Assign the most frequent class as the prediction
6. Repeat the process for all test samples
7. Evaluate performance:
 - a. Compute accuracy
- Accuracy = (Correct Predictions / Total Test Samples)
8. Repeat the experiment for different values of K
Example: K = 1, 3, 5, 7

Tasks to Perform

1. Implement the KNN algorithm as described.
2. Evaluate the model for $K = 1, 3, 5, 7$.
3. Compute classification accuracy for each value of K .
4. Plot K vs. accuracy.

Objective 2

The objective of this offline is to implement the **K-Means Clustering algorithm from scratch** and analyze how the number of clusters (K) affects the clustering results and inertia.

Dataset Preparation

Use the `dataset.txt` in the provided folder.

- The dataset should contain two numeric columns representing 2D data points.

- Load it into a suitable data structure (e.g., list of points or a NumPy array).

Algorithm with Code Hints

1. Set the number of clusters, $K = 4$
Example: $K = 4$
2. Load all data points into a 2D list or array named Data
Example: `data = np.loadtxt("dataset.txt")`
3. Randomly select K points from Data as initial cluster centers
4. Initialize an empty list named Clusters containing K sublists
Hint: `clusters = [[] for _ in range(K)]`
5. Repeat until convergence (centers stop moving significantly):
 - a. For each point in Data:
 - Compute the distance from each cluster center
 - Assign the point to the cluster with the minimum distance
 - b. For each cluster:
 - Update its center as the mean of its points
 - c. Check if the centers have changed very little
 - If yes, stop (convergence achieved)
6. After convergence:
 - a. Compute the final "inertia"
 - Inertia = sum of squared distances from each point to its cluster center
7. Plot the clustered data points with different colors
Hint: use matplotlib's `scatter()` for visualization
8. Repeat for $K = 2, 4, 6, 7$ and record inertia for each case

Tasks to Perform

1. Implement K-Means as described.
2. Run for $K = 2, 4, 6, 7$.
3. Plot the clustered points and their centers for each case.

4. Record and report the inertia for each value of K .

Implementation Guidelines

- Implement the algorithm manually — **do not use sklearn, scikit-learn, or pandas**.
- You may use:
 - `numpy` for numerical operations
 - `matplotlib` for visualization
 - `math` for distance calculations
- Normalize features before distance calculation.
- Use your student ID as the random seed for dataset splitting.

Report and Submission

- Submit both:
 - Your Jupyter Notebook file (`.ipynb`)
 - A short report in `.pdf` format
- The report should include:
 - Plots for each K value
 - A table showing K vs. inertia
 - A brief explanation of how inertia changes with K

Important Notes

- You must follow the given algorithm structure.
- **Copying code** from others will result in a 100% penalty.
- Use of built-in clustering libraries will be penalized.
- Marks depend on both your code and your viva understanding.